

CastagNet

Classification de châtaignes sèches par vision — rapport de projet

Mise en Situation Professionnelle (MESPR) — MSc BIHAR / ESTIA

Auteur : ayala2a · Dépôt : github.com/ayala2a/castagnet

C28 · problématique data

C29 · modèles

C30 · production

C31 · organisation données

C32 · programme IA

Note de méthode. Pour ce projet, je me suis appuyé sur une IA générative (Claude) comme copilote : pour auditer le dataset, retrouver les bonnes références techniques, écrire le code et débbugger. Les décisions, le raisonnement et les choix de direction présentés ici sont les miens ; l'IA a surtout accéléré la recherche et la mise en œuvre.

1. Contexte et problématique

La coopérative GRPTMC exploite une ligne de tri automatique de châtaignes sèches destinées, notamment, à la Farine de Châtaigne Corse — une filière sous signe de qualité où la rigueur du tri conditionne la conformité du produit fini. La machine analyse le flux en continu via **12 caméras** (6 postes, une vue du dessus et une du dessous par poste) et doit classer chaque fruit en quatre catégories : **Conforme, NON Conforme, PIETRA, Vide**. On m'a confié le projet en l'état, avec pour mission de le faire progresser sur trois fronts : la qualité de la donnée, la robustesse du modèle, et la compatibilité avec les contraintes de production.

Le cahier des charges fixe des exigences **asymétriques**, et c'est le fil directeur de tout mon travail : laisser passer un mauvais fruit dans le lot Conforme dégrade la farine et menace la certification, alors qu'écartier à tort un bon fruit ne coûte qu'un peu de rendement. La coopérative demande donc, sur la classe Conforme, une **précision $\geq 95\%$** (peu de mauvais fruits acceptés) et un **rappel $\geq 85\%$** , le tout en **temps réel sur les 12 flux** et sur un matériel modeste (une ancienne carte GeForce GTX 1060 de 3 Go). Chaque décision technique de ce rapport découle de ces contraintes.

2. Le fil directeur

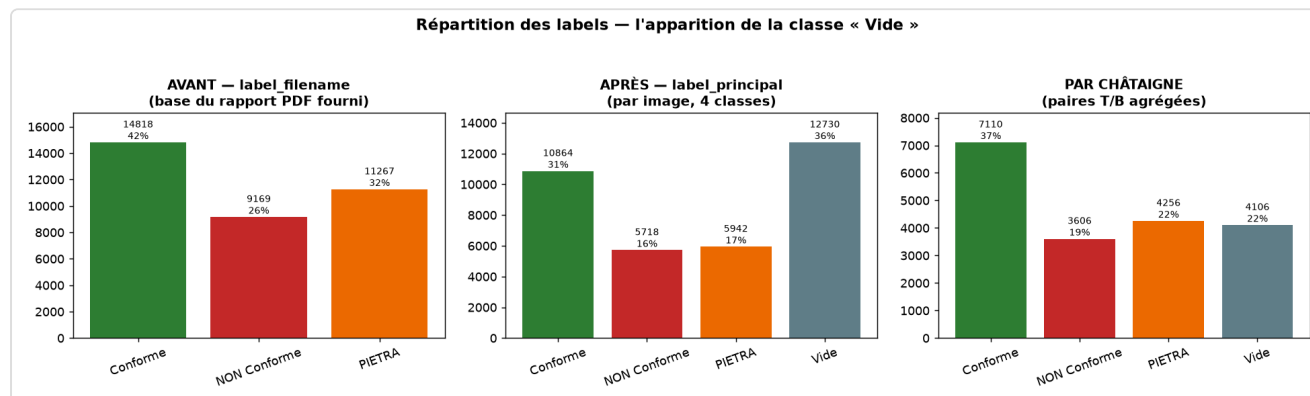
Deux idées structurent tout le projet. La première vient du terrain : **une châtaigne n'est pas une image, c'est une paire d'images** (une vue du dessus, une du dessous). La seconde vient du cahier

des charges : **on préfère toujours écarter un bon fruit que laisser passer un mauvais**. À partir de là, j'ai déroulé le projet dans l'ordre logique — d'abord comprendre et fiabiliser la donnée, ensuite construire un modèle qui décide sur la paire et respecte l'asymétrie précision/rappel, enfin vérifier qu'il tient en production.

3. Travail sur la donnée (§4.1)

3.1 Récupération et audit

Les images (35 254 au total) sont versionnées par DVC et stockées sur un drive partagé — j'ai donc commencé par les rapatrier. En creusant les labels, j'ai découvert un problème important : le rapport statistique fourni ne connaissait que trois classes et ignorait complètement la classe « **Vide** ». En croisant le label déduit du nom de fichier avec la vraie cible relue, j'ai constaté que **35,6 % des images avaient été reclassées en Vide** lors de la relecture (un créneau photographié peut être vide). Le rapport de départ était donc périmé ; je l'ai refait sur les vrais labels.

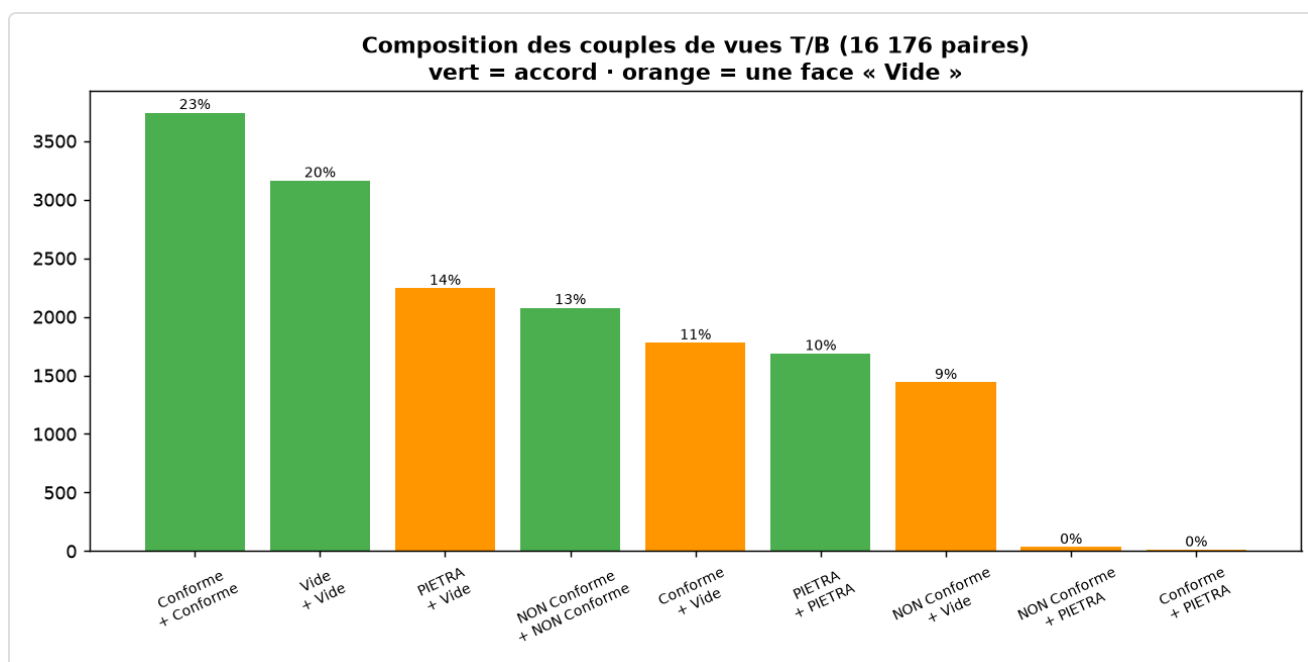


Répartition des labels : le rapport fourni (à gauche) ignore la classe Vide, qui représente pourtant 36 % des images une fois la relecture prise en compte.

J'ai aussi terminé la relecture du reliquat 2026 et remarqué au passage que ce lot était mal étiqueté à l'acquisition (à peine 5 % de vrais PIETRA dedans) — un bon exemple de l'hétérogénéité du diagnostic, que le sujet demandait justement d'analyser.

3.2 Relier les deux vues (dataset)

Puisqu'un fruit = deux photos, j'ai relié les vues T et B via une clé simple : le nom de fichier privé de la position caméra. Cela reconstitue **19 078 châtaignes**. J'ai ensuite analysé les cas où les deux vues étaient en désaccord, et le résultat est net : **100 % des désaccords viennent d'une face vue comme « Vide »** — jamais un vrai conflit entre deux classes.



Composition des couples de vues T/B : les seuls écarts sont « une face Vide », jamais deux vraies classes différentes.

Pourquoi c'est décisif. Cela prouve que décider sur une seule image est trompeur : environ un tiers des fruits présents apparaissent « Vide » sur l'une de leurs deux vues. J'en tire une règle d'agrégation simple et sûre — **la classe non-Vide gagne**, et en cas de vrai doute je prends la plus sévère — qui colle à la priorité « pureté du lot » du cahier des charges.

3.3 Extraction et appariement des vidéos

Deux vidéos de 30 s filment le même créneau, une vue du haut et une du bas, mais avec un micro-décalage temporel. Il fallait les apparier. **Pourquoi ne pas le faire à l'œil** : aligner deux flux de 750 images au jugé n'est ni fiable ni reproductible. J'ai donc mesuré le décalage. J'ai détecté dans chaque vidéo la **séquence d'arrivée des châtaignes**, et constaté qu'elles avaient exactement la même cadence (une châtaigne toutes les 27 images) — donc le même flux. **Pourquoi ce signal plutôt qu'un signal d'image global** : les deux caméras cadrent différemment, et mes premiers essais de corrélation globale donnaient des décalages incohérents (de -102 à +86 images) ; la suite des arrivées, elle, est une information commune et robuste. En alignant les deux séquences, j'obtiens un décalage **net et unique de 5 images (0,2 s)**, la vue du bas en avance. J'ai vérifié une paire à l'écran — même fruit, même repère vert, deux angles — puis sorti une vingtaine de paires propres en prenant, pour chaque fruit, l'image où il est le mieux visible dans chaque vidéo.



3.4 Dataset final et découpage anti-fuite

Pour l'entraînement, l'unité est la **châtaigne** (sa paire), pas l'image. Le découpage train/val/test est **stratifié** (par classe et par année) et surtout **groupé par châtaigne** : les deux vues d'un même fruit tombent toujours dans le même sous-ensemble. **Pourquoi** : sinon la vue T d'un fruit pourrait être en entraînement et sa vue B en test, et le modèle « reconnaîtrait le fruit » (fuite de données) — des scores flatteurs en test mais un échec en production. J'ai vérifié : zéro fuite. Le sous-ensemble vidéo, non labellisé, n'est pas injecté dans l'entraînement (l'ajouter dégraderait le signal) ; il constitue un livrable d'extraction/appariement à part.

4. Modélisation (§4.2)

4.1 Les architectures, et pourquoi

J'ai comparé plusieurs modèles, dans une progression logique guidée par les résultats :

- **Un CNN « maison »** (codé à la main, sur une seule image) comme baseline exigée. Il plafonne et rate presque totalement PIETRA — logique, un défaut ne se voit souvent que d'un côté.
- **Un réseau dual-branch** : deux extracteurs de caractéristiques à poids partagés (siamois) sur les vues T et B, puis une fusion. C'est l'architecture qui correspond au problème (deux vues → une décision). **Pourquoi un backbone léger (MobileNetV3)** : la cible est une GTX 1060 de 3 Go qui doit traiter les 12 flux — il faut rester rapide et compact. **Pourquoi pas de la détection d'objets (YOLO)** : la châtaigne est déjà centrée dans le créneau, on n'a qu'à la classer, pas à la localiser.
- **Une fusion enrichie [T, B, |T-B|]** . En observant que tout plafonnait vers 0,86–0,88 sur mon indicateur clé, j'ai raisonné sur ce que « voit » le réseau : si un défaut n'apparaît que d'un côté, les caractéristiques des deux vues deviennent différentes à cet endroit. Plutôt que de simplement concaténer, j'ai donné explicitement au modèle **l'écart absolu entre les vues**. C'est ce qui a débloqué le rappel.
- **Une représentation radiale** (piste « pour aller plus loin » du sujet) : je déroule le disque en coordonnées polaires avant le réseau. La cohérence radiale s'aligne alors sur un axe, et une rotation du fruit devient un simple décalage — robustesse à l'orientation par construction. C'est finalement mon **meilleur modèle**.

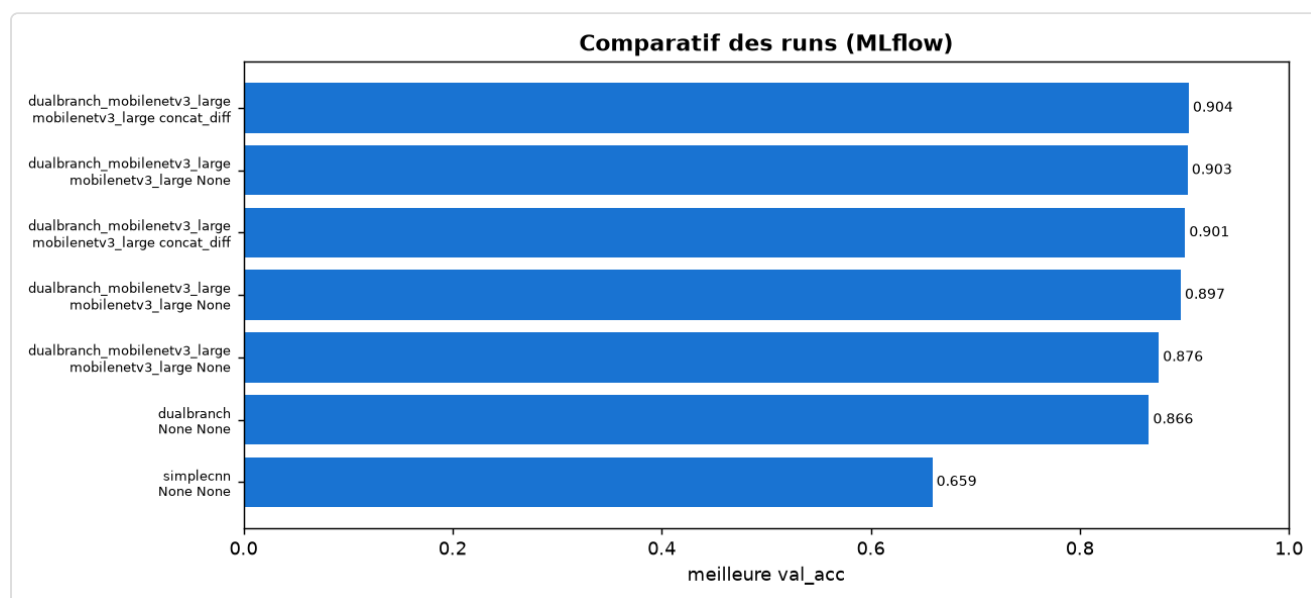
4.2 La métrique de décision, et pourquoi

Je ne me fie pas à l'accuracy globale (trompeuse en déséquilibre) ni à l'argmax brut. Comme le cahier des charges impose une contrainte asymétrique, je **calibre un seuil de confiance** sur la classe Conforme : on n'accepte « Conforme » que si la probabilité dépasse ce seuil. C'est ce

mécanisme qui garantit la précision $\geq 95\%$ tout en maximisant le rappel. J'ai d'ailleurs sélectionné le meilleur modèle non pas sur l'accuracy, mais directement sur le **rappel Conforme à précision 95 %** — l'objectif métier réel.

4.3 Résultats (jeu de test)

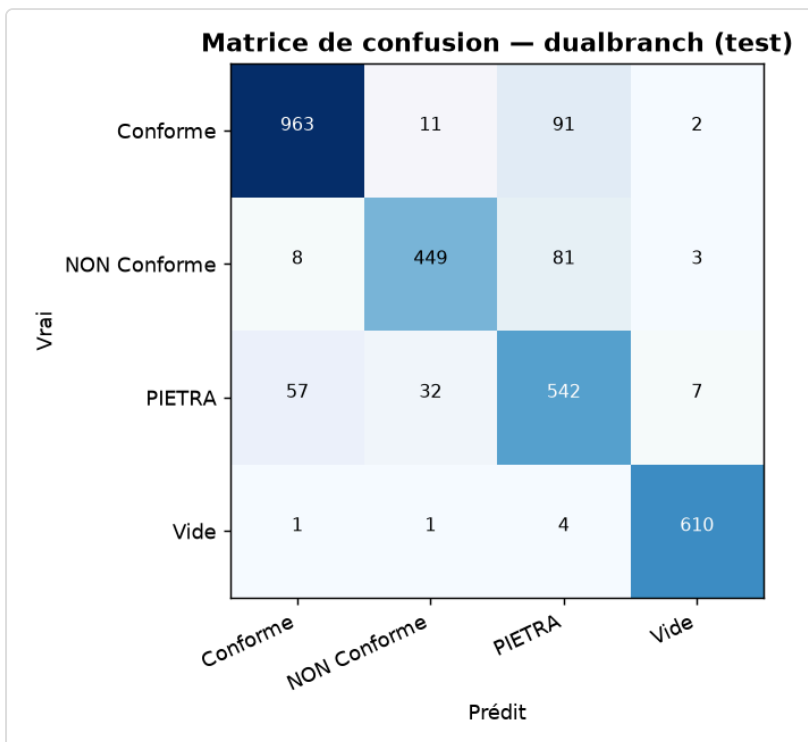
Modèle	Accuracy	Conforme précision / rappel	Cahier des charges
CNN maison (baseline, 1 vue)	0,659	0,954 / 0,296	non
Dual-branch (concaténation)	0,861	0,955 / 0,774	non (rappel)
Dual-branch + fusion IT-BI + TTA	0,913	0,953 / 0,884	oui
Dual-branch radial + TTA (retenu)	0,923	0,951 / 0,925	oui, avec marge



Comparatif des runs suivis dans MLflow.

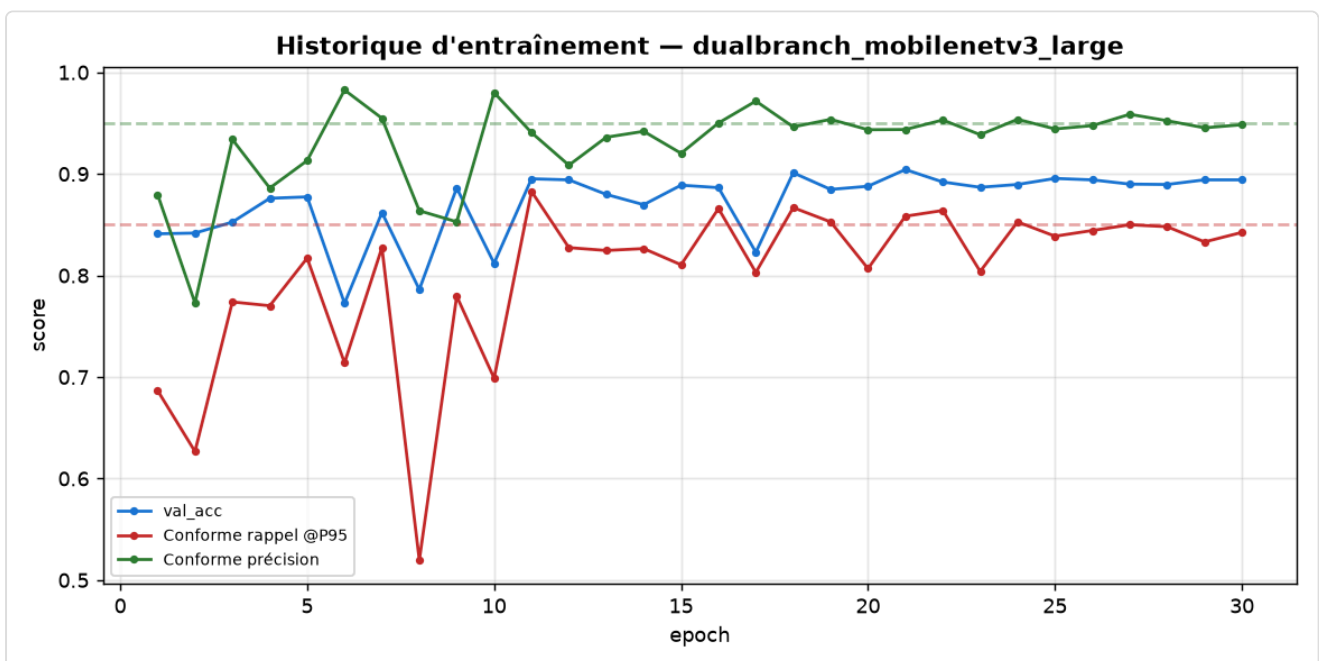
La progression raconte l'histoire : la baseline mono-vue échoue (rappel PIETRA de 0,17, car le défaut ne se voit que d'un côté) ; les deux vues font passer PIETRA à $\sim 0,85$; la fusion **|T-B|** débloque le rappel Conforme au-dessus de 85 % ; et la représentation radiale gagne encore ~ 4 points de rappel ($0,884 \rightarrow 0,925$) à précision équivalente, confirmant l'intuition du sujet sur la cohérence radiale.

4.4 Analyse des erreurs et de l'entraînement



Matrice de confusion du modèle final sur le jeu de test.

Le point dur résiduel est la confusion Conforme ↔ PIETRA (défauts de surface fins). Fait important pour la filière : **les erreurs vont dans le sens sûr** — le modèle recale à tort quelques bons fruits (perte de rendement, tolérée) bien plus qu'il ne laisse passer de mauvais. C'est exactement la philosophie du cahier des charges.



Historique d'entraînement : oscillation initiale puis stabilisation (cosine LR) ; précision Conforme au-dessus de 0,95, rappel autour de la cible.

4.5 Validation croisée (robustesse)

Pour ne pas me fier à un seul découpage, j'ai mené une validation croisée en 5 plis (toujours groupée par châtaigne). Résultats stables : précision Conforme **0,949 ± 0,012**, rappel **0,868 ± 0,019**, accuracy **0,887 ± 0,012**. L'écart-type de 1 à 2 points confirme que le résultat n'est pas un coup de chance. Ces modèles de validation ne sont pas le modèle livré — ils ne servent qu'à en éprouver la fiabilité.

5. Compatibilité production (§4.3)

J'ai exporté le modèle retenu au format **ONNX** (avec un axe de lot dynamique pour traiter d'un coup les images d'un cycle) et vérifié que ses sorties correspondent à celles de PyTorch. Le coût est très faible :

Poste	Valeur
Paramètres	3,71 M
Poids ONNX (FP32)	~15,3 Mo (≈ 7,7 Mo en FP16)
Mémoire au repos / par cycle (6 paires)	~75 Mo / ~220 Mo
Latence par cycle (CPU de dev)	~48 ms (~240 images/s)

Sur les 3 Go de la GTX 1060, la marge est d'un facteur ~10 ; sur GPU en FP16, la latence sera bien plus basse. **Recommandation** : retenir le dual-branch radial + TTA. Toutes les exigences (précision, rappel, temps réel, tenue en 3 Go) sont satisfaites, la mémoire étant le point le plus confortable. L'usage se fait via `predict.py`, qui prend les deux photos d'un fruit et renvoie sa classe.

6. Technologies et ressources utilisées

Outils. Python 3.13 ; PyTorch (accélération MPS sur Mac) et torchvision (MobileNetV3) pour les modèles ; OpenCV pour l'extraction vidéo, le masque circulaire et le déroulé polaire ; scikit-learn (`StratifiedGroupKFold`) pour le découpage anti-fuite ; MLflow pour le suivi des expériences ; ONNX / onnxruntime pour l'export et la mesure de latence ; DVC pour récupérer le dataset ; l'outil Streamlit fourni pour terminer la labellisation ; Git pour le dépôt de travail.

Références qui ont orienté mes choix. Des templates PyTorch de référence pour la structure du projet ; des travaux de tri de fruits/graines multi-caméra (grading de pommes multi-vues, SeedSortNet) qui m'ont conforté dans l'idée du dual-branch ; la bibliothèque timm pour les backbones légers.

7. Conclusion et perspectives

Le projet aboutit à un modèle qui **respecte le cahier des charges avec marge** sur le jeu de test (précision Conforme 95,1 %, rappel 92,5 %, accuracy 92,3 %), tient largement sur le matériel cible, et dont le résultat est prouvé stable par validation croisée. L'apport principal n'est pas d'avoir empilé des couches, mais d'avoir **traduit le problème métier en modèle** : une châtaigne se juge sur ses deux faces, un défaut crée une asymétrie entre les vues, et la représentation radiale exploite la géométrie circulaire du créneau. Piloter chaque choix par la contrainte du cahier des charges — la précision d'abord, sur du matériel modeste — m'a évité d'optimiser un chiffre qui n'aurait pas tenu en production.

Perspectives. Vérifier la part de bruit d'étiquetage dans la confusion Conforme/PIETRA (potentiellement plus rentable qu'un changement d'architecture) ; mesurer la latence réelle sur la GTX 1060 cible ; et envisager un ensemble de modèles si un léger gain de précision était requis, en pesant le surcoût de latence.